

Robofreak Pume

Team

Team members

- Petar Habek (Coder)
- Roko Ivančić (Designer)
- Bruno Maretić (Documentations)
- Viktor Žmegač (Team Captain)
- Ivan Kolarić (our Mentor)

About us:

Our team has lots of experience in the soccer category with many competitions under our belt. We all have multiple years of learning and practicing, and we are excited to learn more through this competition as it is very important to us. We all put a lot of time in this project to do as good as possible in this competition.



Viktor Žmegač
Team Captain

Roko Ivančić
Designer

Petar Habek
Programmer

Bruno Maretić
Documentation builder

Ivan Kolarić
Mentor

Robot

Hardware

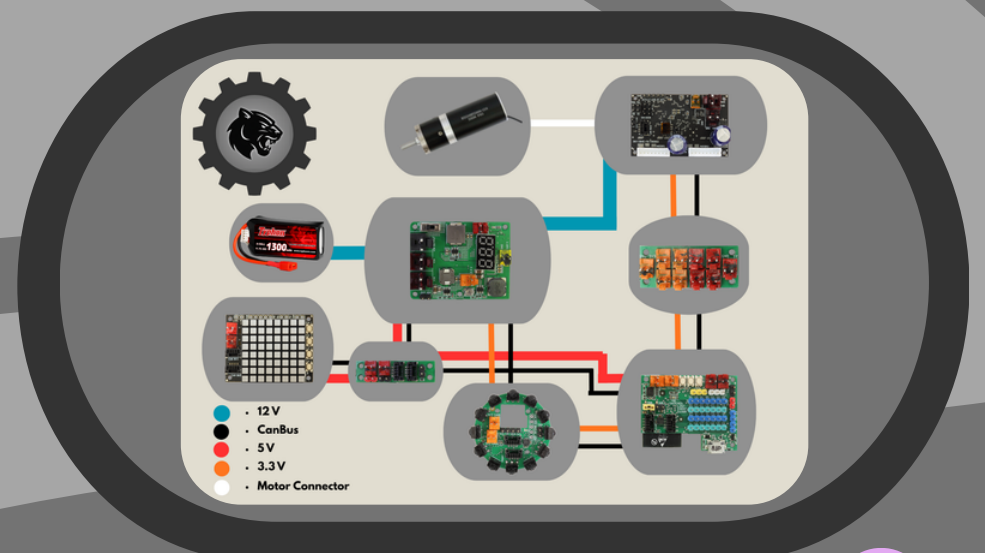
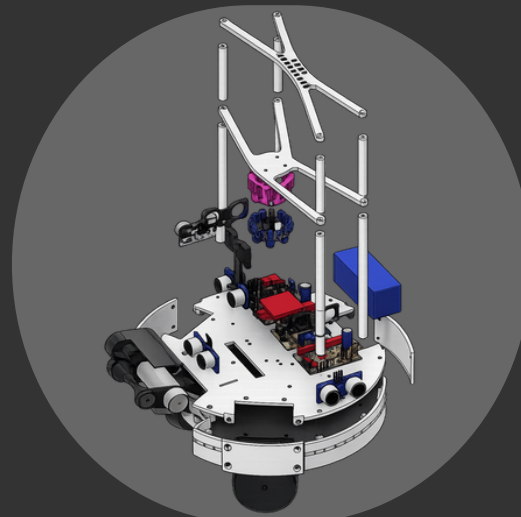
Components:

All of our components are based on the CAN-BUS communication system. Our components consist of:

- MRMS IR Ball Sensor
- Microcontroller ESP-32
- ML-R 8x8 biocolor display
- 3X ML-R BLDC Motor 12 V, 20x40 mm, 1:6.5 (mrm-blde20x40-6.5)
- ML-R Motor Driver
- ML-R Power Supply 3x C, voltmeter
- Typhon 1300mAh 3-Cell 11.1V 30 C Battery

Assembling process:

Most of the assembling was done by our builder Viktor Žmegač and designer Roko Ivančić with some help from other team members. It took us about 3 months to design, build, and program the whole robot. Programming was done by our programmer Petar Habek and team captain Viktor Žmegač, while the documentation was written and made by Bruno Maretić and Petar Habek.



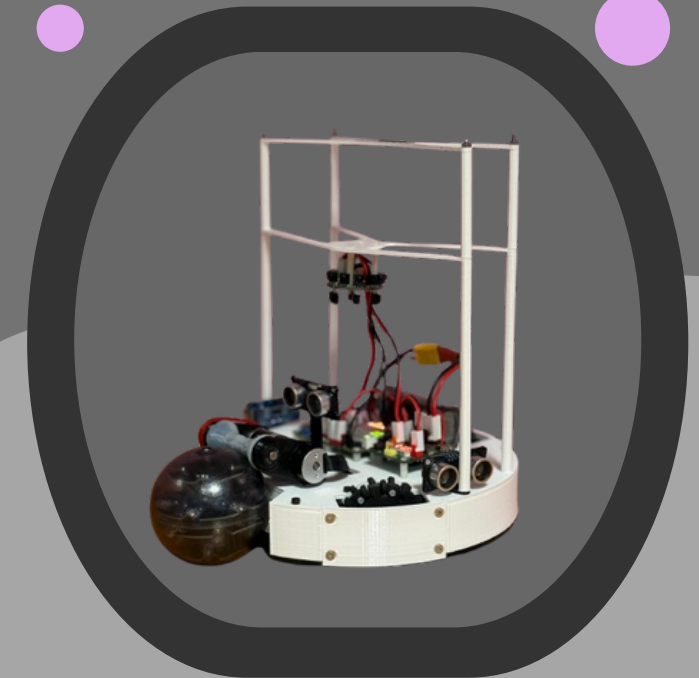
Design

Our designer Roko Ivančić did most of the work designing the chassis, the handle and the wheels all by himself. We designed everything in Autodesk Fusion 360. Our parts that are printed on a 3D printer are made out of PLA. The chassis consists of lower circular base, side covers/panels and upper plate. The handle is mounted on top of the upper chassis. Side covers/panels are mounted in front of the wheels so that if it hits a wall or other robot, the wheels won't disassemble.

The electric components are assembled on top of the upper chassis because if they were inside, they would overheat. Our addition and the part we worked the most on was the dribbler. It consists of the same motor that we use for wheels, 3 gears with different ratios and the rubber that gives the robot more grip on the ball. The dribbler is also not fixed to the chassis, but it is pushed down by its weight so when the ball goes in and lifts up the dribbler, the robot has even more grip on the ball as it is slightly pushing it down.

The ball sensor is mounted high because:

- it gives a better view of the surroundings
- it can detect the ball more easily
- the robot parts do not block its field of view
- a higher position often provides more accurate distance and direction measurements
- It is safer from other robots hitting it



Software

```
void RobotSoccer::tri(int speed, int direction, int rotation){
```

Our function 'tri' tells our robot how to orient itself with looking at the position of the ball. It basically tells our robot where to go, at what angle to be looking at (depending on the compass) to follow the ball.

```
float headingOutput = -pidRotation->calculate(headingError);
```

This is our formula that calculates the angle our robot should be looking at with the compass and PID system implemented in our robot's system. It keeps track of our robot's angle so it doesn't lose control.

The Program:

Our program is made in program language Arduino C++. The robot is programmed to mainly just follow and get the ball in the dribbler when the ball is in front of it, and go around the ball when it's in front of it. While that is happening, the robot is constantly fixing its heading so it's always facing the enemy's goal.

The robot could also position itself using ultrasonic sensors. If ultrasonic sensors were placed on the front, back, left, and right sides, the robot could measure distances from the walls and calculate its position on the field.

PID is a control algorithm used to keep the robot stable and accurate. It stands for Proportional, Integral, and Derivative. The PID controller constantly compares the desired position or angle with the current one and corrects the motors to reduce the error. In this program, PID is mainly used for rotation and movement correction.

